

Module 5 | Per-user protection scheme

Introduction

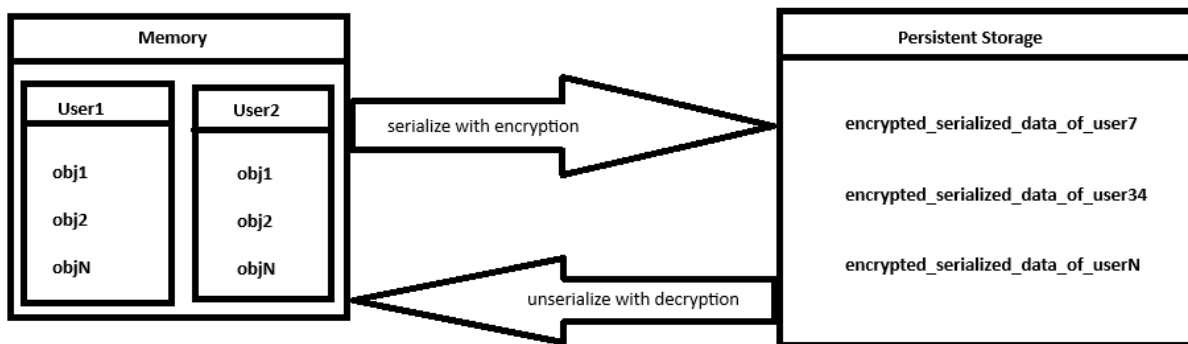
In a system with total of 1000 potential users among which no more than 20 are active at any time, with total of 50,000 objects where each user has an average of 200 permitted objects, the goal is to maintain a directory per user listing allowed objects efficiently. Since no more than 20 users are active at a time, the protection mechanism should be optimized for fast checks with low memory usage while still being able to scale to all users. The given scenario and the proposed scheme is very relatable to how modern operating system works and manages things.

Design Considerations

Given the scenario, each user can access only 200 out of 50000 objects. i.e. around 0.4%. Thus, using memory as a storage for the 50000 objects is waste of memory. It is worth noting that the objects in this context may represent things like a file, folder, program, network resources and so on. We can optimize the system by storing active-only user's data in memory, and inactive user's data on persistent storage like a harddrive or something similar.

Proposal

For the actual implementation of the proposed scheme, we can maintain each user's objects in a hash set. With hash set, we get data lookup of $O(1)$ as we can directly retrieve objects occupied by a user by using the user's id. The pseudocode of hash set looks something like this: {user_id: [obj1, obj2, ..., obj200]}. Thus, hash set is very fast for checks and accesses while occupying maximum memory of 800 bytes per user and 16 KB for all the active users combined. i.e. $200 * 4 \text{ bytes_per_object_id}$. Additionally, we can use global object map for storing object specific data from an object's perspective. This helps us to store additional information like attributes (locks, status) of the objects and know which users can access a given object by maintaining a reverse mapping. E.g. {obj1: {attr: any, users: [user1, user2, userN]}}.



For inactive users, their directories are kept on disk in serialized form (simple file structure, JSON, binary) and brought/unserialize back into memory only when the user logs in. For safety, the serialized and memory data can be encrypted using the user's login credential. And since at most 20 users are active at any time, memory consumption is tiny while access checks remain fast and constant. Thus, adding more users and objects only increases the size of the disk storage.

Conclusion

This scheme is efficient because of the very nature of limited number of users being active with limited objects to use. Access checks also remain fast using the hash set, and memory use stays minimal even with thousands of users. This scheme aims to balance speed, scalability and storage efficiency, making it well suited for systems with many users only handful active at any given time.